```
Output:
int(-1)
int(1)
```

Here we can see that with a natural comparison, "M4" would be ordered before "M18" (because the entire numbers are compared). Note that the "M40" and "M18" comparison returns 1, and not 3. The nat functions only return -1, 0, or 1; they do not return the difference between non-matching bytes in equal-length strings.

A quick word on safety with these functions: All of the str*cmp() functions are binary safe, as the following demonstrates:

```
var_dump(strcmp("abc\x00a", "abc\x00z"));
var_dump(strnatcmp("abc\x00123", "abc\x00125"));
var_dump(strcasecmp("ABC\x00123", "abc\x00125"));
var_dump(strnatcasecmp("abc\x00M4", "abc\x00M11"));
```

```
Output:
int(-25)
int(-1)
int(-2)
int(-1)
```

Multi-byte safety is a trickier concept with these functions. Consider the following:

```
$mbStr1 = "\u{141}ukasz";
$mbStr2 = "Zebedee";

print "{$mbStr2} and {$mbStr1} are out of order." . PHP_EOL;
var_dump(strcmp($mbStr1, $mbStr2));
```

```
Output:
Zebedee and Łukasz are out of order.
int(107)
```

Now, the two strings aren't equal, clearly, but strcmp() is reporting that "\u{141}ukasz" is greater than "Zebedee" by a value of *107*. Well, this is true if we look at byte values:

```
print bin2hex($mbStr1) . PHP_EOL;
print bin2hex($mbStr2) . PHP_EOL;
```